December 6, 2025
CMSC 129

Legolas Tyrael Lada
Sharmaigne Angelie Mabano
Aaron Dave Siapuatco

Custom Programming Language

Introduction

1.1 Purpose

The purpose of this software is to illustrate a functioning Integrated Development Environment (IDE) and compiler for the custom programming language, known as IOL or integer-oriented language.

1.2 Scope

Currently, the system can handle source code editing, file management (New File, Open, Save, Save As), lexical analysis, syntax analysis, semantic analysis, as well as program execution.

1.3 Technology Stack

The project utilizes the Python language, and utilizes only the standard libraries to run and use this software. While there are other libraries in the repository, it is **only used for development purposes**, and not essential for running the project.

System Design and Implementation

2.1 Architecture Overview

The system follows a modular compiler architecture integrated into a GUI or the IDE. The data flow proceeds as follows:

1. Source Code Loading
2. Lexical Analyzer
3. Token Stream
4. Syntax Analyzer
5. Semantic Analyzer
6. Interpreter

Moreover, we utilize this grammar and Extended Backus-Naur Form (EBNF)

None

```
| Category               | Token Type          | Lexeme / Pattern
| Program Delimiters     | IOL                 | IOL
|                        | LOI                 | LOI
| Data Types             | INT                 | INT
|                        | STR                 | STR
| KEYWORD                | INTO                | INTO
|                        | IS                  | IS
|                        | BEG                 | BEG
|                        | PRINT               | PRINT
| Operators              | ADD                 | ADD
|                        | SUB                 | SUB
|                        | MULT                | MULT
|                        | DIV                 | DIV
|                        | MOD                 | MOD
| Built-in Commands      | NEWLN               | NEWLN
| Literals               | INT_LIT             | A sequence of digits
| Variables              | IDENT               | Starts with a letter
| Special                | EOF                 | (End of File)
| Error                  | ERR_LEX             | Any error
```

None

EBNF

```
Program ::= 'IOL' Statements 'LOI' 'EOF'
Statements     ::= { Statement }
Statement              ::= VarDecl
                         |      Assignment
                         |      Input
                         |      Output
                         |      Newline
VarDecl        ::= 'INT' Ident [ 'IS' Expression ] | 'STR' Ident [ 'IS' Ident ]
Assignment     ::= 'INTO' Ident 'IS' Expression
Input          ::= 'BEG' Ident
Output         ::= 'PRINT' Expression
Newline        ::= 'NEWLN'
Expression   ::= IntLiteral
               | Ident
               | MathOp Expression Expression
MathOp       ::= 'ADD' | 'SUB' | 'MULT' | 'DIV' | 'MOD'
Ident        ::= Letter { Letter | Digit }
IntLiteral   ::= Digit { Digit }
Letter       ::= 'a'...'z' | 'A'...'Z'
Digit        ::= '0'...'9'
```

2.2 Lexical Analysis

Here, is where the code scans the raw source code loaded from the editor, and converts it into a stream of tokens, also known as tokenization.

- **Token Output:** The scanner generates a *.tkn* file containing the token stream
- **Token Definitions:**
    - **INT_LIT:** Integer Literals
    - **IDENT:** Variable identifiers
    - **Keywords:** Reserved words such as IOL, LOI, IS, INTO, BEG, PRINT, ADD, SUB, MULT, DIV, MOD, NEWLN
    - **ERR_LEX:** Used for unknown words not recognized by the language.

2.3 Syntax Analysis (Parser)

The parser receives the token stream and verifies that the sequence of tokens adheres to the grammar.

- **Parsing Method:** Recursive Descent
- **Expressions:** Used Prefix Notation

2.4 Semantic Analysis

The system also performs type checking during the parsing phase to ensure operations are valid.

- **Type Safety:** Ensures that math operations are only on INT types.
- **Declaration Check:** Verifies that variables are defined before use
- **Symbol Table:** A table is maintained to track variable names and their values.

2.5 Execution

Once lexical, syntax and semantic analyses are performed, the execution runs. Which will produce an output that displays literals, variable values or expression result in the IDE's console.

User Manual

3.1 Getting Started

1. Have Python installed on your machine
2. Run `python main.py`

3.2 User Interface

The IDE is divided into three main sections

1. **Code Editor:** The large text area for writing IOL code.
2. **Console:** The bottom panel displaying compilation status, errors and a program output.
3. **Variable Table**: The side panel that shows the active variables and their types.

3.3 Features

- **File Menu**
  - **New File:** Creates a blank text area for a fresh script.
  - **Open File:** Opens an existing *.iol* file
  - **Save/Save As:** Saves the existing written code in the text area.
- **Compile**
  - **Tokenize:** Tokenizes the source code in the text area.
  - **Show Tokenized Code:** Displays the tokens, or the output of "Tokenize"
- **Parsing**
  - **Success:** "Analysis Successful. <file_name>.iol compiled with no errors found" appears.
  - **Failure:** "Compile Unsuccessful", then a list of errors and their line numbers are displayed.
- **Execute:** After a successful, lexical, syntax and semantic analysis, the code should run with no errors, and output the result in the console.

3.4 Sample Run

LINK:

https://drive.google.com/file/d/1qkqYBOtEMPEKVD6zCooDJEWJ34HP2W-9/view?usp=sharing

1. Open an existing *.iol* file, or select New File and type the a iol compatible code.

2. Save the current code using File > Save or File > Save As
3. Then click Compile > Tokenize, this should prompt a 'Tokenization Complete" message in the console.
    a. Optionally to check the tokenized code, go to Compile > Show Tokenized code, the tokenized code should now be shown in the console.
4. After a successful tokenization, go to Execute.
5. Results will be shown in the console below:
    a. For a successful run, it will show the expected output with no errors.
    b. For an unsuccessful run, it will show the errors with the line, and word where the error message was found.

Task Distribution

The project was equally divided among the three members, which included, grammar and EBNF making, lexical analysis, syntax analysis, semantic analysis, execution, and the proper documentation.

Legolas Tyrael Lada: Implemented the UI and its integration, the lexical analyzer and helped with the documentation.

Sharmaigne Angelie Mabano: Implemented the Syntax and Semantic Analysis and created the EBNF.

Aaron Dave Siapuatco: Implemented the interpreter, displayed the table of variables, created the documentation, grammar rules and the demo video.